Lab 2: Fine-tune vs prompt

Does fine-tuning beat your best prompt?

Aarne Talman

5 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	**3 / 5 Nov**	**Post-training**	**Lab 2: tune vs prompt**	**Aarne**
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

The question

One question

Lab 2

For *your* task, does fine-tuning beat your best prompt?

That is the whole lab. Everything today is here to help you answer it **with numbers you produced yourself**.

Tuesday's lecture gave you the machinery: assistant-only loss, LoRA, catastrophic forgetting, and the decision slide (*behaviour → fine-tune, knowledge → retrieve*). Today you test it on your own task.

What week 2 adds to your project

A prompting baseline and a fine-tuned model, **both scored on the same held-out set**.

You may serve the adapter you train today in week 3, or use another compatible model. Week 4 examines evaluation in more depth.

Start with `Week-2/Lab-2/README.md`; it contains the baseline, LoRA, evaluation and smoke-test commands.

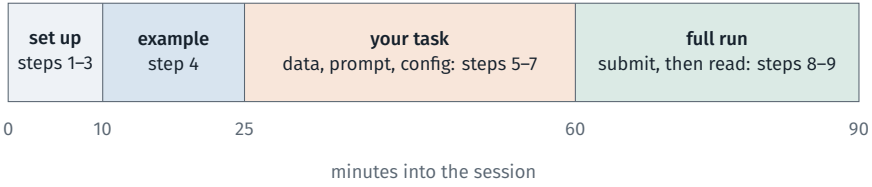
Material for the final report

Setting	Held-out score	What it is
Zero-shot		best of the base or instruct model
Best prompt		choose on dev; report held-out score
Fine-tuned (LoRA SFT)		same held-out set, same decoding
Forgetting check	before, after	50 general-knowledge questions

Plus one loss curve, with training *and* evaluation loss on the same axes.

Plus one paragraph: was it worth it? GPU-hours and human hours spent, against points gained. Would you ship it?

Today, minute by minute



This briefing stops at minute 10. If the smoke test has not passed by minute 15, put your hand up.

Steps 1–9 refer to `Week-2/Lab-2/README.md`.

The full experiment is **one batch job** that runs the baselines, LoRA training and the forgetting check in sequence, and it can take an hour or more. Submit it by minute 60, even with a first version of your prompt. You can improve the prompt and run again later.

PROTOCOL

Run four baselines

	Zero-shot	Few-shot (5 examples)
Base model		
Instruct model		

Compare all four on the **development set**, with the **same decoding parameters** and the **same answer-extraction function**.

Whichever scores highest on dev is *the* baseline. Report its test score only after you have made that choice. Do not report the instruct zero-shot number just because it was the easiest one to run.

A weak baseline fakes a win

Beating a bad prompt tells you nothing. If you have not tried few-shot on the instruct model, you do not have a baseline yet.

Decoding settings and the held-out set

Fix these once, use them everywhere

temperature	0 (greedy)
`max_new_tokens`	long enough for your longest gold answer
seed	fixed and recorded
prompt template	identical across all four
answer extraction	one shared function

Greedy decoding is the right choice here because the task has one correct answer and you want the four runs to be comparable. (Week 3: temperature 0 is *not* deterministic once requests are batched. Today it is close enough.)

Three splits

- **train**: the SFT data, 1–5k examples
- **dev**: where you choose prompts, epochs, learning rate
- **held-out test**: touched once per reported number

Take your few-shot examples from **train or dev**, never from test. Report the split sizes and the metric in the table caption. “Accuracy” on an unnamed subset is not a number anyone can use.

Three ways to silently ruin a baseline

1. Answer extraction. The model answered "She sold 72 clips." Your parser wanted a bare integer. Score: 0. *Fix:* one extraction function, shared by every run, and ten raw generations read next to their extracted answers (`show_examples.py`).

2. A chat template on a base model. The base model never saw `<|im_start|>` or any other control token. Those arrive in post-training. *Fix:* plain completion-style prompts for the base model, `apply_chat_template` only for the instruct model.

3. Contamination. GSM8K, MNLI and MMLU are in everyone's pretraining mix, so part of your "zero-shot" score may be recall. *Fix:* you cannot decontaminate someone else's pretraining, so say so in the report. Week 4 treats this properly.

Check the parser before you believe the number

A score of exactly 0.0 is a reason to inspect the extraction code before concluding that the model failed. Before you believe *any* baseline number, read ten raw generations beside their extracted answers: `show_examples.py --run <name>`.

Parser and template errors often make the baseline too low. Contamination can make it too high. Either error invalidates the comparison.

Doing it

Your data: one conversation per example

```
# data/task/train.jsonl: one labelled example per line
{"id": "train-001", "premise": "A dog sleeps.",
 "hypothesis": "An animal rests.", "label": "entailment"}

# lab2lib/prompting.py turns it into one conversation for TRL
def messages_for_sft(record):
    return [
        {"role": "system",    "content": TASK_SYSTEM},
        {"role": "user",      "content": task_prompt(record, [])},
        {"role": "assistant", "content": f"Label: {record['label']}"}]
```

- For your task, change two settings in `prompting.py`: `TASK_FIELDS` (your input fields) and `TASK_SYSTEM` (the instruction). The model's own chat template is applied for you.
- The loss is computed on **assistant tokens only**, so the reply is the training target while the prompt remains context.
- Start with **1–5k examples**. The assistant turn is always `Label: <label>`, the format the scorer expects.
- Create the splits with `make_splits.py`, check them with `check_data.py`, and read the rendered prompts in a mock run before you use a GPU (README, steps 5–7).

Starting hyperparameters, and the three you may tune

Knob	Start at
LoRA rank	16
LoRA alpha	32 (twice the rank)
LoRA dropout	0.05
target modules	`all-linear`
learning rate	0.0002
schedule	cosine, 3% warmup
epochs	2
effective batch	16 sequences (8×2)
max length	1024 tokens
precision	bf16

You may tune exactly three

1. **learning rate**
2. **number of epochs**
3. **LoRA rank**, the capacity of the adapter (move alpha with it, keeping alpha at twice the rank)

Leave everything else alone. Change *one* knob at a time and log every run in one small table: knob, value, train loss, eval loss, held-out score.

That table is more useful than a better score without a record of what changed.

Source: Common LoRA practice and TRL/PEFT defaults. These are **starting points**, so do not quote any of them in the report as a finding.

When the loss falls and nothing else moves

Lower training loss is not a result

Training loss down, evaluation loss flat or rising, held-out score unchanged: you memorised the training set. With LoRA this happens *fast*,

because you have more trainable parameters than training tokens.

Do the arithmetic for your own run.
`model.print_trainable_parameters()` gives you the numerator. Your token count is roughly (examples) $\times$ (tokens per example).

A plausible shape for a 1–4B model with rank 16 on all linear layers:

trainable parameters	$\sim$10–40 M
2k examples $\times$ 300 tok	$\sim$0.6 M tok

What to do about it

- plot **train and eval loss together**, always
- stop at the epoch where eval loss turns up
- then reduce epochs, then the learning rate, then *r*, in that order

Source: Parameter counts here are an order-of-magnitude illustration. Print the real number for your own model and report that.

The forgetting check

50 general-knowledge questions, unrelated to your task. An MMLU slice is fine.

Score them **twice**: on the model you started from, and on the fine-tuned model. Same script, same decoding, same extraction.

Treat this as a **regression test**. Nobody cares whether your small model knows 42% or 55% of MMLU; what you are reporting is the **difference** between before and after.

A few points down is normal and worth one sentence. Twenty points down means the adapter ate the model: fewer epochs, lower learning rate, lower rank.

Why this is in the lab at all

Tuesday's claim: *always measure what fine-tuning broke, not only what it improved.* GLM-5 ends its pipeline with self-distillation precisely to recover capabilities weakened by the earlier RL stages. Catastrophic forgetting turns up as a line item in production recipes.

50 questions is a blunt instrument

With only 50 right/wrong items, one standard error is about **7 percentage points** when accuracy is near 50%. Report the pair of numbers and do not over-read a 3-point change.

Source: MMLU-as-regression-test framing: Raschka, *Understanding the 4 Main Approaches to LLM Evaluation.* Standard error from the binomial, $n = 50$.

Before you leave

The report table and files to keep

Setting	Score	Config
Zero-shot, instruct		greedy
Few-shot 5, instruct		greedy
Zero-shot, base		greedy
Few-shot 5, base		greedy
LoRA SFT		rank 16, 2 epochs
Forgetting, before		50 Q
Forgetting, after		50 Q

Caption it with: dataset, split sizes, metric, model IDs.

Keep for the final submission

- your config, prompt and scoring code
- `results.json` with every number above
- `predictions/`, the raw generations
- the loss curve (train *and* eval)
- the LoRA adapter, or its path on Roihu if it is large

Keep the adapter if you want to serve it in week 3. Optional DPO and GRPO extensions are described in the README.

Roihu reminders

The lab reservation

The reservation covers roughly **16:00–20:00**, so jobs started in the session can finish after it.

Follow a job with `squeue --me` and `tail -f logs/lab2-lora-<jobid>.out`.

Next scheduled maintenance: **1–2 December**.

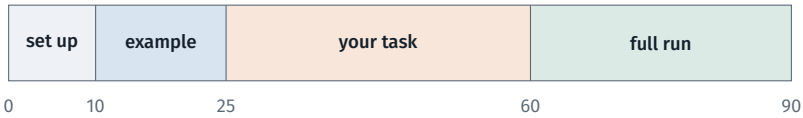
Etiquette, unchanged all term

- `gputest` (15 min, short queue) before `gpumedium`
- **close idle GPU Jupyter sessions.** One forgotten session holds a whole GH200 for twelve hours, and we share one allocation
- never train on a login node; it has no GPU and its CPUs are shared
- `seff <jobid>` after every batch job, to see what you actually used

Environment: build and test the course environment on Roihu's ARM login nodes before requesting a GPU. The training configuration uses bf16, which the GH200 supports natively.

Source: CSC Roihu documentation: system description, partitions, course maintenance windows.

Leave this on the wall



- [ ] smoke test passed (step 2)
- [ ] example job finished on `gputest` (step 4)
- [ ] splits made and `check_data.py` clean (step 5)
- [ ] prompt adapted and read in a mock run (steps 6–7)
- [ ] full job submitted to `gpumedium` (step 8)
- [ ] table printed with `summarize_results.py`
- [ ] ten raw generations read with `show_examples.py`
- [ ] loss plot checked, results and config saved

Stuck for more than ten minutes? Put your hand up. The README has a troubleshooting table.

Appendix: things that will go wrong

Score is exactly 0 or 100, or many `None`
Read the raw outputs: `show_examples.py --run <name>`. Do the labels in `TASK_SYSTEM`, the config and the data match?

`KeyError: 'text'` (or another field)
`TASK_FIELDS` does not match your JSONL fields. Run `check_data.py`.

`ModuleNotFoundError: trl` (or `peft`)
Repeat step 3: `module load`, then `pip install --user`.

`CUDA out of memory`
In the config, halve `per_device_train_batch_size`, double `gradient_accumulation_steps`, or reduce `max_length`.

Job stuck in `PD` (pending)
`squeue --me --start` for an estimate. Check the account and partition; `scancel <jobid>` to cancel.

No log file
`logs/` does not exist: `mkdir -p logs` and resubmit.

Job stops at the time limit
Use `gpumedium` with a longer `--time`, or smaller dev and test sets.

Loss is `nan`
Lower the learning rate. Everything runs in bf16.